UNIVERSITÄT HEIDELBERG
ZUKUNFT SEIT 1386

aip

Neuroscience-inspired Artificial Intelligence

Cognitive Science - Heidelberg University Lecture Series

Winter 2023/24

Prof. Dr. Artur Andrzejak

January 8, 2024

Artificial Intelligence for Programming (AIP)
https://aip.ifi.uni-heidelberg.de

Motivation and Overview

Why Neuroscience-inspired AI?

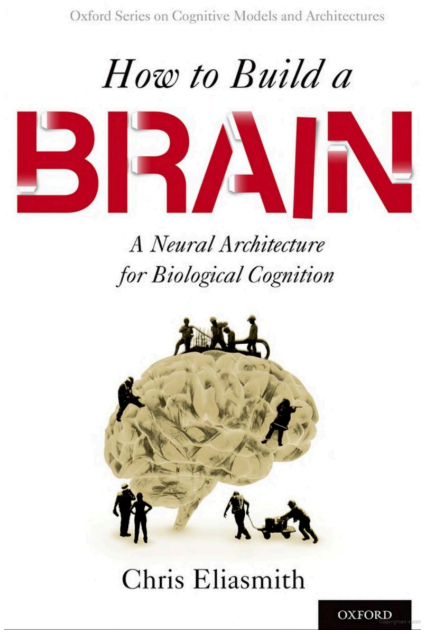
- Human brain is a proof that intelligence is possible at all
- Modelling it is an alternative or a complement to creating AI “from scratch”
- Unfortunately, we do not really know how the brain works
  - We even do not even understand the mechanics of some fundamental processes, e.g. learning, memory, reasoning, …
- However, understanding nervous systems of simpler organisms like **drosophila melanogaster** (fruit fly) is more advanced
- We can use this knowledge to create building blocks of AI systems
  - General principles: e.g. sparse distributed representations, noise tolerance, …
  - Concrete algorithms: e.g. FlyHash, continual learning, …

Different Levels of Abstraction

- Modeling biological intelligence is a multi-level problem
  - From the level of **neurons** to the level of **behavior** (and beyond)
  - The distinction is the proximity to the biological “hardware” (neurons, synapses, …)
- There are many interesting projects/approaches at different levels of abstraction
  - From **neurons**: Blue Brain Project, Human Brain Project, …
  - To **brains structures**: SPAUN (Semantic Pointer Architecture Unified Network), Nengo, …
  - To **principles** and **algorithms**: VSAs, sparsity, FlyHash, …
- Today we will focus on the **principles** and **algorithms** level because it is the most practical one
  - Lower abstraction levels do not (yet) scale to real-world problems

Example of the Level Brain Structures

- Projects: Neural Engineering Framework, Semantic Pointer Architecture Unified Network (SPAUN), How to Build a Brain
  - Chris Eliasmith at Computational Neuroscience Research Group, University of Waterloo
- Book: Chris Eliasmith, **How to Build a Brain: A Neural Architecture for Biological Cognition**, 2013 (EI13)



Aspects Covered in This Lecture

- **Examples of successful transfers** from neuroscience to AI
  1. Solutions to fundamental algorithmic problems uncovered by studying drosophila and their generalizations
     - **FlyHash**, (**FlyModel** for **continual learning**)
  2. General principles of the representation and processing of information in the brain
     - **Hyperdimensional computing** (aka VSAs), (**sparsity**)
- **Discussion of the differences** between biological and AI systems
  - What are the differences on the level of observable behavior?
  - Which current “building blocks” of AI are biologically implausible?
  - => Which AI mechanisms are missing or should be improved?

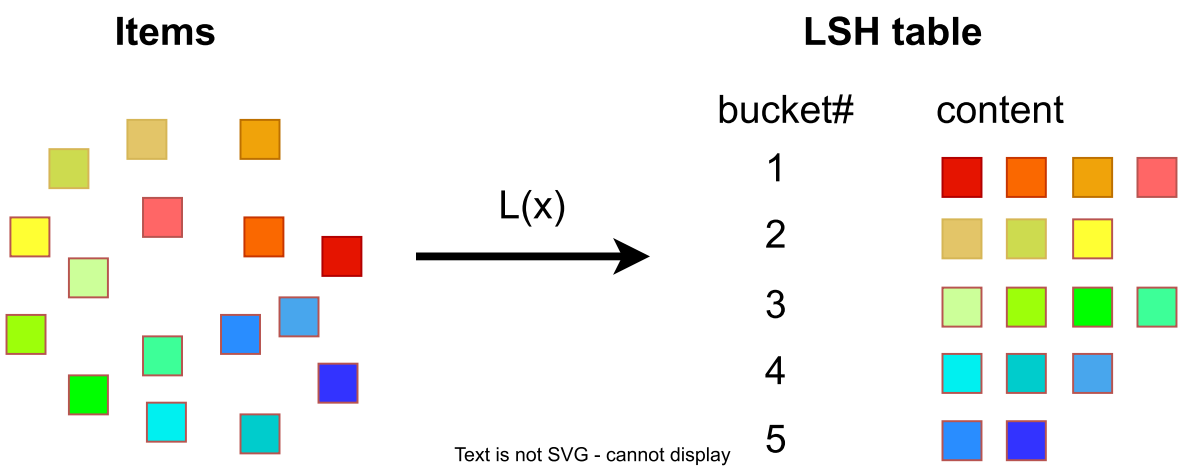
FlyHash: A Neuroscience-inspired Algorithm for Similarity Search

Similarity Search (Nearest Neighbor Search)

- **Similarity search** is the task of finding similar items in a collection
  - E.g. similar images in a database or similar documents on the web
- It is a fundamental computing problem, especially relevant for large-scale information retrieval systems
  - Think of Google search, Amazon product recommendations, …
- A trivial solution is to compare a query item with each collection item
  - But this is not feasible for large collections (e.g. millions of images, billions of documents) of high-dimensional items (e.g. images)
  - Also not energy efficient (e.g. for mobile devices)

Similarity Search via Locality Sensitive Hashing

- Luckily, there is a class of algorithms that can solve this problem efficiently: **Locality Sensitive Hashing** (LSH)
  - LSH algorithms map items to a set of **buckets** (or **bins**) such that similar items are mapped to the same bucket with high probability
  - => Similarity search is reduced to finding similar items in a bucket



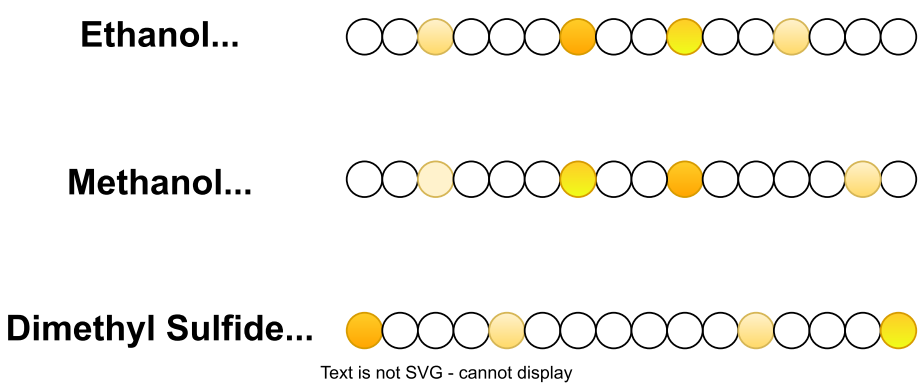
- There are many variants of LSH algorithms, e.g. **MinHash**, **SimHash**, **Random Projection Hashing**, **Multi-Probe LSH**, …

FlyHash Algorithm

- In 2017, a new variant of an LSH algorithm was proposed: **FlyHash**
  - Found by studying the neural circuits of a **fruit fly olfactory system**
  - Sanjoy Dasgupta et al., **A neural algorithm for a fundamental computing problem**, Science 2017 (Da17)

Fruit Fly Olfactory System /1

- Fruit can learn to associate odors with rewards or punishments
- Their olfactory circuits generate a **“tag”** for each odor
  - The tag is a set of neurons that fire when that odor is present
  - These neurons are called **Kenyon cells** (KC) (about 2,000 KCs)
- Interestingly, this tag is a **sparse high-dimensional vector**
  - Only about 5% or 100 neurons out of 2,000 fire for a given odor
- Similar odors generate similar tags (see ethanol vs. methanol)

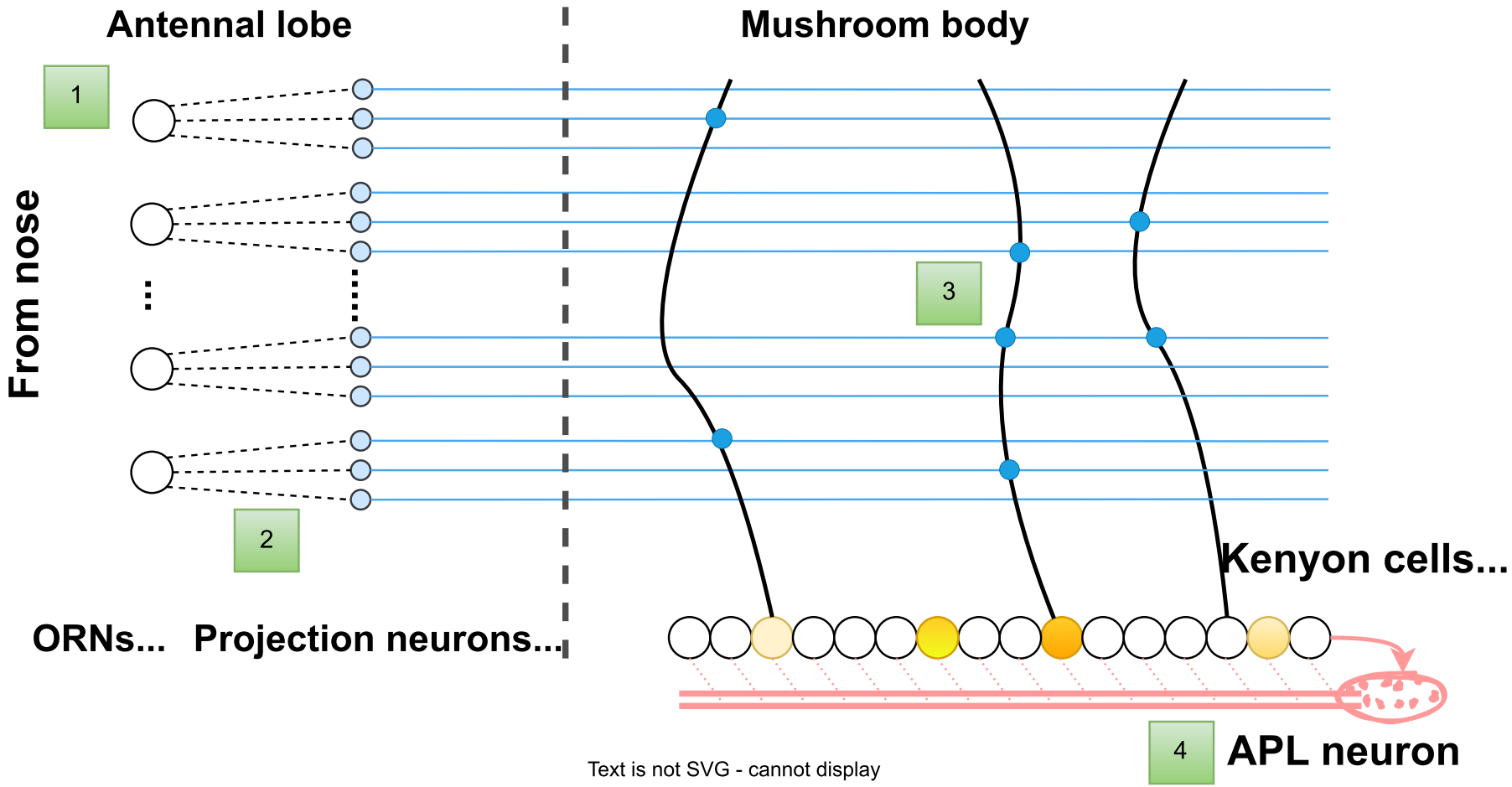


Fruit Fly Olfactory System /2

The tags are generated as follows:

1. The odor is detected by **olfactory receptor neurons** (**ORNs**)
   - ~50 types, each with different sensitivity and selectivity to odors
2. ORNs are connected to **projection neurons** (**PNs**)
   - ~150, these remove dependence of the signal on concentration
3. PNs connect to **Kenyon cells** (**KCs**) in a sparse and random way
   - Each of the KCs receives input from about 6 different ORNs
   - => Dimensionality is expanded from ~50 to ~2,000
4. Sparsity of the tags is achieved by a single inhibitory neuron called **APL** (**anterior paired lateral** neuron)
   - All but the 5% most active KCs are silenced by APL
   - It is a **Winner-Take-All** (**WTA**) circuit

Fruit Fly Olfactory System /3



From Fruit Fly to FlyHash

- The fruit fly olfactory system is a **biological** LSH algorithm
  - It maps odors to sparse high-dimensional vectors (tags) such that similar odors are mapped to similar tags
- The key elements are:
  - **Random projections** of the inputs (ORNs) to outputs (KCs)
  - **Expansion** of the input space (d~50) to a high-dim. space (d~2,000)
  - **Sparsity** of the output (tags) via a WTA circuit (APL)
- Based on this principles, a new **computational** LSH algorithm was proposed: **FlyHash**
  - It maps items to sparse high-dimensional vectors (tags) such that similar items are mapped to similar tags (LSH property)
  - Similar performance as state-of-the-art LSH algorithms, but requires about 20x less computations

FlyHash Algorithm

Input: vector $\boldsymbol{x} \in \mathbb{R}^d$

Parameters: hash length m, sparsity factor k, sampling rate α

Generate mk sparse, binary random projections
$S = \{S_i \mid S_i = \{\lfloor \alpha d \rfloor$ random integers in $[0, d]$ $\}\}$

# Compute high-dimensional projection $P(\boldsymbol{x})$ (dimension mk)
for $j = 1$ **to** mk **do**

  $P_j(\boldsymbol{x})$ = sum of $\boldsymbol{x}$ components at indices S_j

end for

Sparsify the result - retain m largerst entries

$h_1(\boldsymbol{x}) = \mathrm{WTA}(P(\boldsymbol{x}), m)$

return $h_1(\boldsymbol{x})$

Fruit fly: we have $d = 50$, $\alpha = \frac{6}{50}$, $k = 20$, $m = 100$ => the output is a vector $h_1(\boldsymbol{x})$ of dimension $mk = 2000$ with $m = 100$ non-zero entries

Binning with the FlyHash Algorithm

- Remaining difficulty: since the tags (output $h_1(\boldsymbol{x})$) is high-dimensional, even similar input vectors are unlikely to have same tags (i.e. be mapped to the same bucket)
- For example, for the data set SIFT-1M (1 million images) with $k = 20$ and $m = 16$ (tags have dimensionality $mk = 320$), FlyHash produces about 860,000 unique tags
  - => almost all images are mapped to different buckets
- A solution is a so-called **multi-probe LSH**, where nearest neighbors of a query are searched also in “nearby” buckets
- This approach adapted for FlyHash is detailed in the follow-up paper:
  - Jaiyam Sharma and Saket Navlakha, **Improving Similarity Search with High-dimensional Locality-sensitive Hashing**, 2018, (Sh18)

Learning in AI: Backpropagation

- ML/AI: **backpropagation** (BP) compares a correct output with the actual output of the model, and updates the weights over the whole model computes
  - Based on the gradient of the error with respect to the weights of the model (i.e. how much does each weight contribute to the error)
- BP is biologically implausible, e.g. because it requires backlinks to every “storage location”, and is computationally expensive
- Geoffrey Hinton (Turing-Award 2018), one of the inventors of backpropagation and deep learning, repeatedly attempted to replace BP by other learning mechanisms
  - E.g. Forward-Forward Algorithm from 2022 (Hi22)

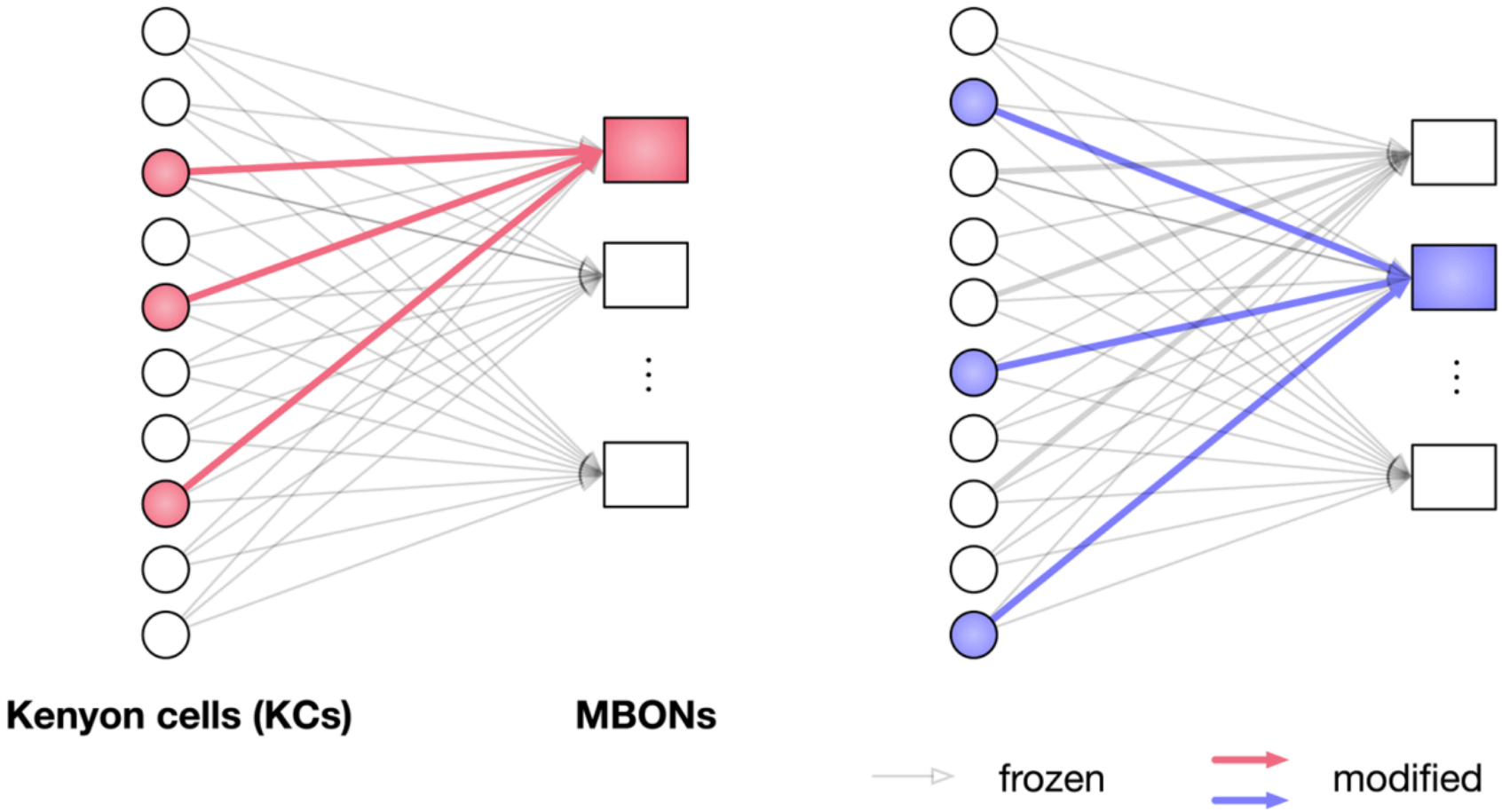
Learning in Biological Organisms: Fruit Fly, again! /1

- Fruit fly olfactory circuitry uses another mechanism based on **associative learning** (Mo20)
  - It connects odors (as high-dimensional tags discussed before) with rewards/punishments
- The fly olfactory system is a **continual learning** system, i.e. it can learn new associations without forgetting old ones
  - I.e. it can learn new associations in arbitrary order, without forgetting old ones
  - Contrary to current ML/AI, which requires shuffled training data
- Btw: this mechanism was also successfully transferred to AI as the **FlyModel** (Sh23)

Learning in Biological Organisms: Fruit Fly /2

- Fruit fly stores associations between odors and behaviours in synapses between the **KCs** (**Kenyon cells**) and the **MBONs** (**Mushroom Body Output Neurons**) (Fe18, Sh23)
- How does fly associates an odor A with an electric shock?
  - Initially, synapses from odor activated Kenyon Cells (KCs) have equal weights to both the “approach” MBON and the “avoid” MBON
  - Pairing odor A with punishment weakens **only** the synapses between the “approach” MBON and the KCs
    - This is achieved via releasing dopamine to a compartment containing only such synapses
  - **All other synapses remain “frozen”**, especially the synapses between the “avoid” MBON and the KCs
  - Over time, the reduced synaptic weights between odor A KCs and the approach MBON reliably establish the avoidance association

Learning in Biological Organisms: Fruit Fly /3



From: Yang Shen, Sanjoy Dasgupta, Saket Navlakha; Reducing Catastrophic Forgetting With Associative Learning: A Lesson From Fruit Flies. 2023, (Sh23)

Learning in Biological Organisms: Fruit Fly /4

- There are multiple differences between this approach and backpropagation
- Learning is **highly localized** to only the synapses between the KCs and the MBONs
  - Low energy cost, also mechanistically interpretable
- Because we learn to associate to simultaneous events, there is no need for backlinks or “error computation”
- No changes of already correct associations (e.g. different to the perceptron algorithm)
  - Experiments show that this is essential for **continual learning**

Lessons Learned from the Biological Paragon

- The fruit fly LSH algorithm uses a **combination** of random projections, high dimensionality, and sparsity
- Used separately in computational LSH algorithms, but not together
- Why binary, sparse random projections?
  - **Energy efficiency**: computational savings of factor 20 vs. dense, Gaussian random projections
- Why sparsity?
  - **Fault tolerance**: if a few neurons fail, the tag is still similar to the original one
  - Supports **learning** of associations between odors and rewards/ punishments => FlyModel by Y. Shen et al. (2021)
- Why high dimensionality?
  - A general principle of neural computation => **HD computing**

Computing with High-Dimensional Vectors

Basic Concepts

Traditional Computing vs. Brain Computing

- What **computers** are good at, but brains are not
  - Raw speed
  - Fast and accurate arithmetic
  - Following instructions literally

- What **brains** are good at (and energy efficient as well)
  - Recognizing people and things
  - Learning from examples, reason by analogy
  - Learn to use language and reason by logic
  - Control our interaction with the world

Computing with Huge Vectors: Vector Symbolic Architectures (VSAs)

- **Fan-in** and **fan-out**: number of input/output signals entering/leaving a circuit
- Neuroscience studies observed that nervous systems have huge fan in's/fan out's: **On the order of 1,000 – 200,000 nerves**
- This motivated investigation of data processing with huge vectors: dimensions 1,000 – 10,000
- Umbrella terms:
  - **Vector Symbolic Architectures** (VSAs, R. Gayler)
  - **Hyperdimensional Computing** (HDC, P. Kanerva)
  - **Semantic Pointer Architecture** (SPA, C. Eliasmith)
- Considered as a "closest shot" at brain-like AI technology

Why are VSAs Interesting? /1

- VSAs use **distributed representation** to encode information
  - Concepts or symbols are represented by patterns of activity across a large number of elements (vectors), similarly as in the brain
- **Hierarchical and uniform representation of aggregates**
  - VSAs can represent combinations of concepts and/or their sequences as a single vector => individual concepts and their collections are represented in the same way
- VSAs can use **sparse representation** (only few entries are non-zero)
  - => Noise tolerance: If one neuron fails, the representation of the concept can still be maintained by the remaining neurons
- **Generalization**: Similar concepts can share overlapping representations, allowing for the generalization of knowledge

Why are VSAs Interesting? /2

- **Symbolic** methods: explicit algorithms, rule-based systems with "hard" knowledge, (logic) programming languages,…
- **Sub-symbolic** methods: neural networks, probabilistic methods, …

Symbolic	Syb-symbolic
Symbols	Numbers
Logical	Associative
Serial	Parallel
Reasoning	Learning
von Neumann machines	Dynamic Systems
Localised	Distributed
Rigid and static	Flexible and adaptive
Concept composition and expansion	Concept creation and generalization
Model abstraction	Fitting to data
Human intervention	Learning from data
Small data	Big data
Literal/precise input	Noisy/incomplete input

Why are VSAs Interesting? /3

- VSAs can bridge **symbolic** and **sub-symbolic** representations and methods
- Example: heterogenous learning methods:
  - [Sub] Learning from data via **backpropagation**
  - [Sym] Learning **holistic transformations** (functions manipulating data structures) from only 1 example (or few ones)
  - [Sym] **Manually engineering functions** for manipulating symbolic and continuous data
- Different options, but same framework
  - => Usable in a unified system

How to Compute with HD Vectors?

- A few components and few operations make a surprising powerful system of computing
- Components
  - **Representational space**, e.g. binary/complex vectors
  - **Operations** on these representations (4 key operations)
  - **Item memory** (or associative memory)
  - **Measure of similarity** $\mathrm{sim}()$ between vectors (distance-based)
- Mandatory properties
  - High-dimensionality, typically $d = 1000...10,000$
  - Randomness

Representational Space: Options

- **TPR**: Tensor Product Variable Binding
  - Paul Smolensky, 1989

- **(F)HRR**: (Fourier) Holographic Reduced Representation
  - Tony Plate, HRR: 1990, Fourier: 1995
  - C. Eliasmith: Semantic Pointer Architectures (SPA)

- **BSC**: Binary Spatter Codes
  - Paul Smolensky, 1989

- **MAP**: Multiply Add Permute
  - Ross Gayler, 1998

Representational Space: Options /2

- Less common representations:
  - **MBAT**: Matrix Binding of Additive Terms
  - **SBDR**: Sparse Binary Distributed Repr., Kussul et al.
  - **SBC**: Sparse Block-Codes, M. Laiho et al.
  - **GAHRR**: Geometric Analogue of Holographic Reduced Representations, D. Aerts et al.
- More information and a historical overview:
  - Denis Kleyko, Module 2 at Seminar **Computing with HD Vectors**, UC Berkeley, 2021, (Kl21)

VSA Operations

- **Bundling** or **superposition** or **"add"**: $x + y$
  - For creating sets
  - E.g. via component-wise addition
- **Binding** or **multiply**: $x \otimes y$
  - For "labeling" a vector/value x with y or vice versa
  - E.g. via component-wise multiplication
- **Unbinding** or **release**: $\otimes\, x^{-1}$
  - Essentially, an inverse of binding: $(y \otimes x) \otimes x^{-1} = y$
  - For retrieving a labelled value
- **Permutation** or **"protect"**: $\rho(x)$
  - For creating sequences
  - E.g. cyclic shift of a vector

Example 1 of a VSA Representation: MAP

- **Multiply / Add / Permute (MAP)** representation (Ross Gayler, 1998)
- Vectors are "bipolar", i.e. their elements are $+1$'s and -1's
  - Essentially correspond to binary $0/1$-vectors
- E.g. three random vectors with dimensionality $d = 10,000$
  - $A = (-1 + 1 - 1 + 1 + 1 + 1 - 1 \ \ldots \ +1 - 1 - 1)$
  - $B = (+1 - 1 + 1 + 1 + 1 - 1 + 1 \ \ldots \ -1 - 1 + 1)$
  - $C = \underbrace{(+1 - 1 + 1 + 1 - 1 - 1 + 1 \ \ldots \ +1 - 1 - 1)}_{10{,}000 \text{ dims}}$
- Each vectors represents a **symbol**, e.g. "cake", "red", "man"

Multiply Add Permute (MAP) - Operations

- **Superposition**
  - Component-wise addition $x + y$ = $(x_1 + y_1, x_2 + y_2, ..., x_d + y_d)$
  - Depending on the variant may be normalized or not
- **Binding**
  - Component-wise multiplication $x \otimes y$ = $(x_1 y_1, x_2 y_2, ..., x_d y_d)$
- **Unbinding**
  - Same as binding, i.e. component-wise multiplication
- **Permutation**
  - Any (random but fixed) permutation of the vector components

Example 2 of a VSA Representation: FHRR

- **Fourier Holographic Reduced Representations (FHRR)** (Tony Plate, 1995)
- **Representational space**: Vectors have as components complex numbers $e^{i\varphi}$ with $\varphi \sim U(-\pi, \pi)$
  - i.e. φ is drawn from the uniform distribution in $[-\pi, \pi]$
- **Superposition**: Component-wise addition with normalization (to have unit magnitude)
- **Binding**: Component-wise multiplication $\odot$
- **Unbinding**: Component-wise multiplication with the complex conjugate of the binding vector
- **Permutation**: Any (random but fixed) permutation of the vector components

Similarity of Vectors

- **MAP**: Similarity $\mathrm{sim}(x, y)$ between two vectors x and y is measured by the **dot product**
  - I.e. sum of component-wise products: $\mathrm{sim}(x, y) = x \cdot y = \sum_{i=1}^{d} x_i y_i$
  - Essentially, not normalized **cosine similarity**

- **FHRR**: Similarity is **mean of sum of cosines of angle differences**:

$$\mathrm{sim}(x, y) = \frac{\mathrm{Real}(x^T \cdot y^*)}{d}$$

Similarity of Random HD Vectors

- For two random vectors x and y with a large number of dimensions, the expected value of their similarity is nearly 0:

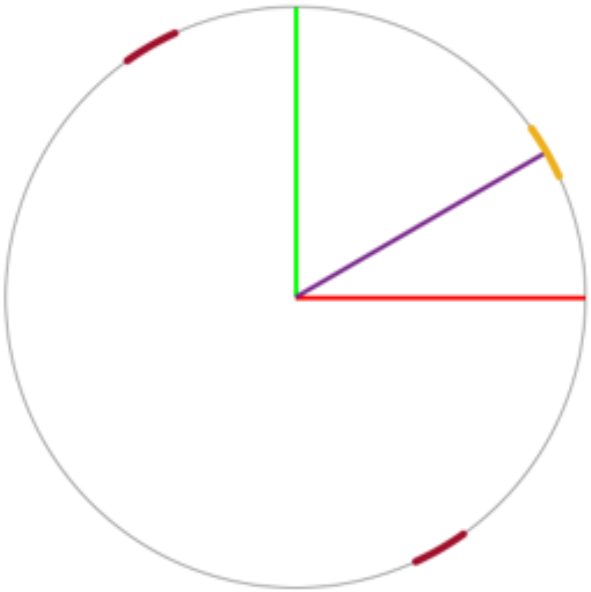
$$E[\mathrm{sim}(x, y)] \approx 0$$

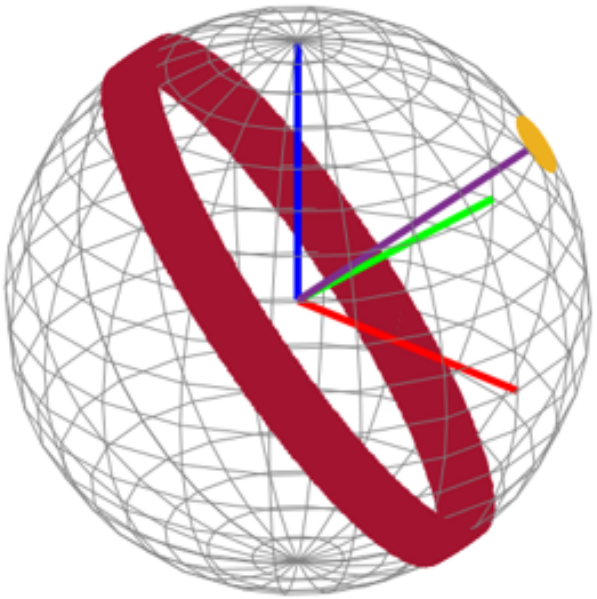
Key property of high-dimensional vectors:

Almost any two of such vectors are dissimilar, approximately orthogonal!

Similarity of Random HD Vectors: Intuition

Given a random vector x in R^2 or R^3, what are the sets of vectors **dissimilar** (or **approximately orthogonal**) to x?

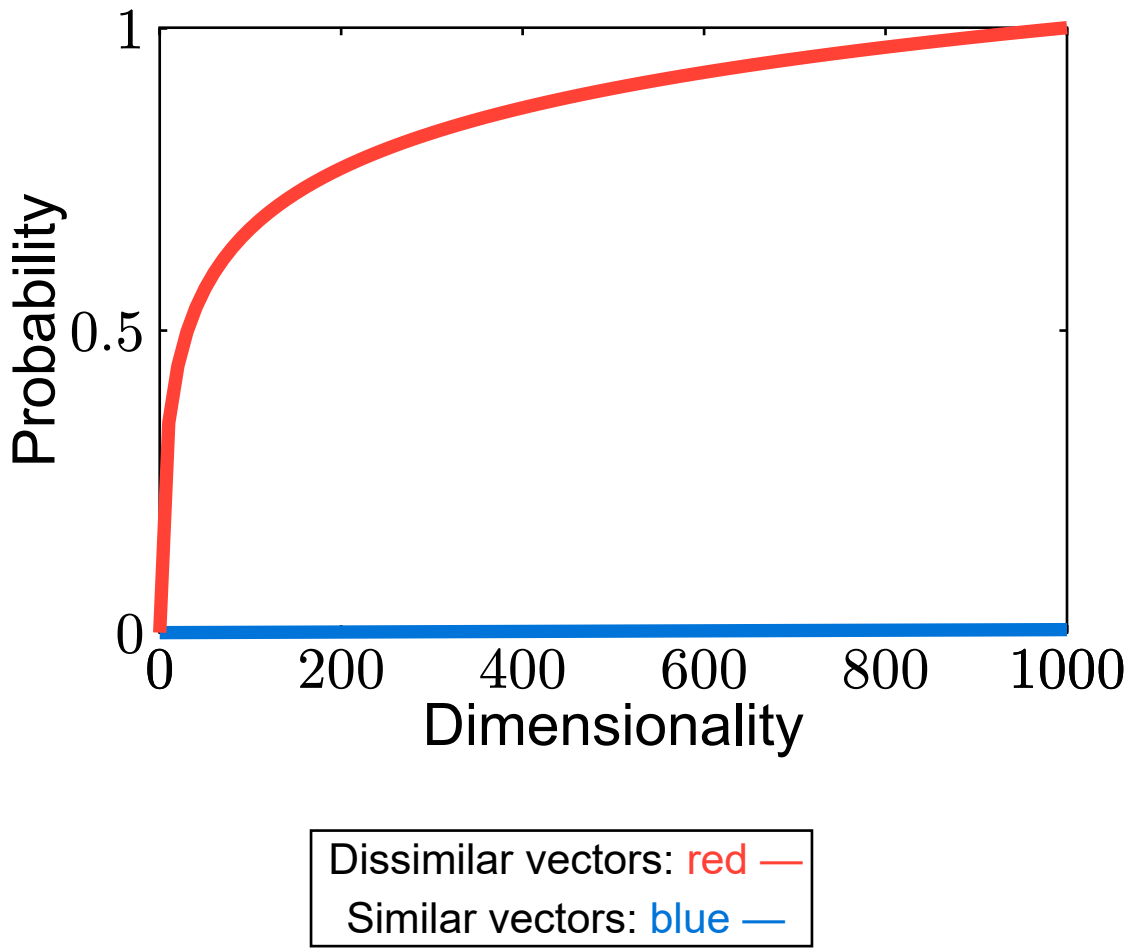




From: Peer Neubert et al., An Introduction to Vector Symbolic Architectures and Hyperdimensional Computing, ECAI 2020, (Ne20)

Similarity of Random HD Vectors: Intuition

Probabilities that two random vectors are almost orthogonal (red) vs. similar (blue) by the number of dimensions



Properties of VSA Operations in Terms of Similarity

- **Bundling** or **superposition** or **"add"**: $+$
  - Sum of VSA vectors is **similar** each of the summands: $\mathrm{sim}(x+y, x) \approx 1$ and $\mathrm{sim}(x+y, y) \approx 1$
  - => We can easily test for set membership
- **Binding** or **multiply**: $\otimes$
  - "Product" of VSA vectors is **dissimilar** to each of the factors: $\mathrm{sim}(x \otimes y, x) \approx 0$ and $\mathrm{sim}(x \otimes y, y) \approx 0$
  - => Essentially, we can label and "protect" a vector
- **Permutation** or **"protect"**: $\rho(x)$
  - As with binding, the result is **dissimilar** to the argument: $\mathrm{sim}\big(\rho^k(x), x\big) \approx 0$ for any $k > 0$
  - Allows to label an element with its position k in a sequence, and distinguish it from other positions of same element

Data Structures with VSA Vectors

Traditional vs. VSA-based Data Structures

- VSA vectors can be used to represent “collection-like” data structures
  - E.g. records, lists, trees, graphs, …
- Let us store a record with 3 fields x, y, z (values are a, b, c):
  - $h = \{x : a, y : b, z : c\}$
- Traditional computing
  - Each x, y, z is a named field (= offset address), i.e. $h.x$, $h.y$, $h.z$ are different storage locations
  - So each a, b, c is a value stored at a **separate** offset address
  - Of course, h needs more memory than each of a, b, c
- As we will see shortly, with VSAs we store a record h in a **holographic**, **superposed** way
  - … as a single vector of the **same** dimensionality as each of a, b, c!

VSA-based Records

- With VSAs, we store record h in a superposed way, without fields:
  1. The first part is the HD vector H representing the record h: $\boldsymbol{H = x \otimes a + y \otimes b + z \otimes c}$ (recall that $\otimes$ is the "bind" operation)
  2. The second part is an item memory storing all active HD vectors, especially a, b, c, x, y, z
     - For cleaning up after retrieval (comes later)
- Note that H is a **superposition** of three HD vectors $x \otimes a$, …, $z \otimes c$ and has the same dimensionality as each of a, b, c
- **=> This gives a uniform representation of individual values and their collections**
- However, in total we do not save memory (in this case), because we need the item memory

Retrieving from VSA-based Records /1

- Given a query q and a record $H = x \otimes a + y \otimes b + z \otimes c$, how to retrieve the value labeled with e.g. $q = x$?
  - Traditional computing: let res = $h.x$
- In VSAs, we have two steps: (1) unbind, and (2) clean up

1. Unbinding for $q = x$ (we assume MAP, as unbind and bind operations are the same, i.e. each element is its own inverse under $\otimes$)

$$
\begin{aligned}
x \otimes H &= x \otimes (x \otimes a + y \otimes b + z \otimes c) \\
&= x \otimes x \otimes a + x \otimes y \otimes b + x \otimes z \otimes c \\
&= a + \text{noise} + \text{noise} \\
&= a' \approx a
\end{aligned}
$$

Retrieving from VSA-based Records /2

2. Clean up the result a' (which we expect to be $\approx a$)
   - Find among the vectors stored in the item memory M a nearest neighbor to a' (i.e. vector with the highest similarity to a')
   - If done right: $\mathrm{sim}(a', a)$ is significantly higher than $\mathrm{sim}(a', v)$ for any other vector $v \neq a$

Observations /1

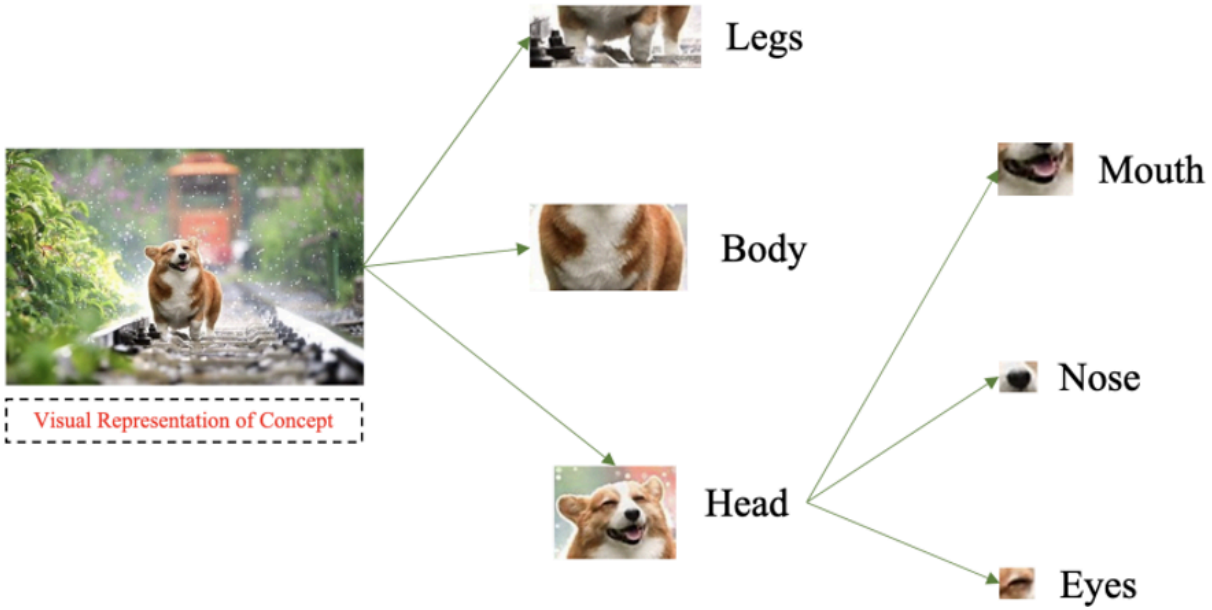
- Why are the noise terms in not a problem for retrieving?
  - $$\begin{aligned} x \otimes H &= x \otimes (x \otimes a + y \otimes b + z \otimes c) \\ &= x \otimes x \otimes a + x \otimes y \otimes b + x \otimes z \otimes c \\ &= a + \text{noise} + \text{noise} \end{aligned}$$
- Because a binding operation $\otimes$ produces a vector which is **dissimilar** to any of the arguments:
  - $\text{sim}(p, p \otimes r) \approx 0$ and also $\text{sim}(p, p \otimes r \otimes s) \approx 0$ for any p, r, s
  - This is a key property of the binding operation $\otimes$
- So, in particular, $\text{sim}(b, x \otimes y \otimes b) \approx 0$ and $\text{sim}(c, x \otimes z \otimes c) \approx 0$
- => The 2nd and 3rd terms will not interfere: They won't erroneously give us values of b or c

Observations /2

- Why can we reliably clean up, i.e. a is a vector in M with highest similarity sim to a'?
- Recall: Almost any pair of random HD vectors is dissimilar (approximately orthogonal)
- In particular, each of x, y, z, a, b, c stored in M are mutually approximately orthogonal
- => a + noise + noise will be most similar to a and not to any other vector in M

Hierarchical Data Structures

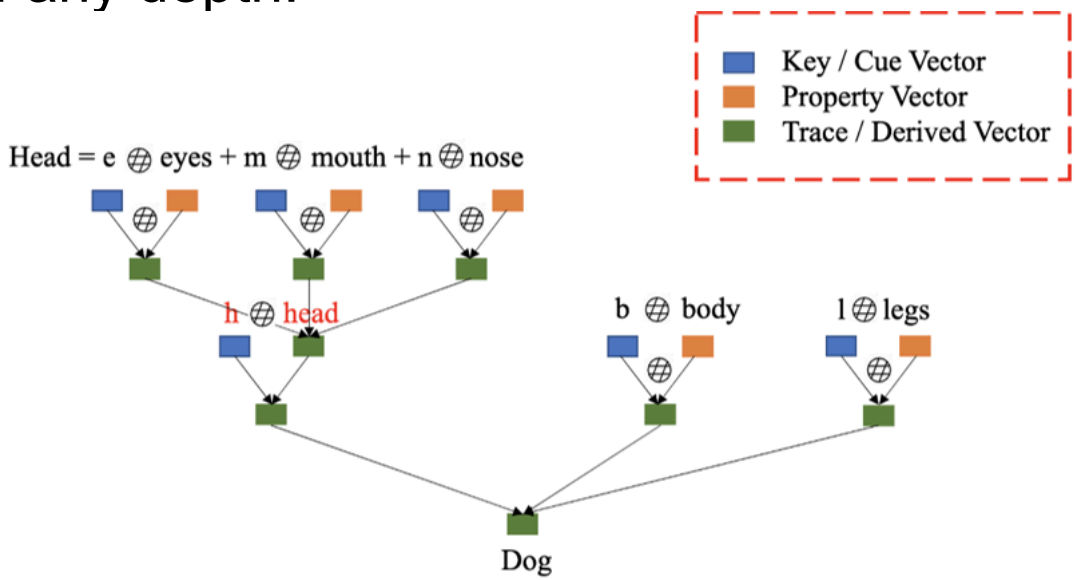
- VSAs can be used to store (arbitrarily deep) hierarchies of data
- Example: Concept dog as hierarchy of different parts



From: Ashwinkumar Ganesan et al., “Learning with Holographic Reduced Representations”, 2021, (Ga21)

Representing Hierarchies

- Encode dog as a superposition vector:
  - $dog = h \otimes head + b \otimes body + l \otimes legs$
  - h, b, l are HD-vectors representing key (fields)
  - $head$, $body$, $legs$ are values, e.g. “pointers” to image parts
- $head$ is itself a superposition of “deeper” parts
- Note: all vectors have the same dimension => We can create hierarchies of any depth!

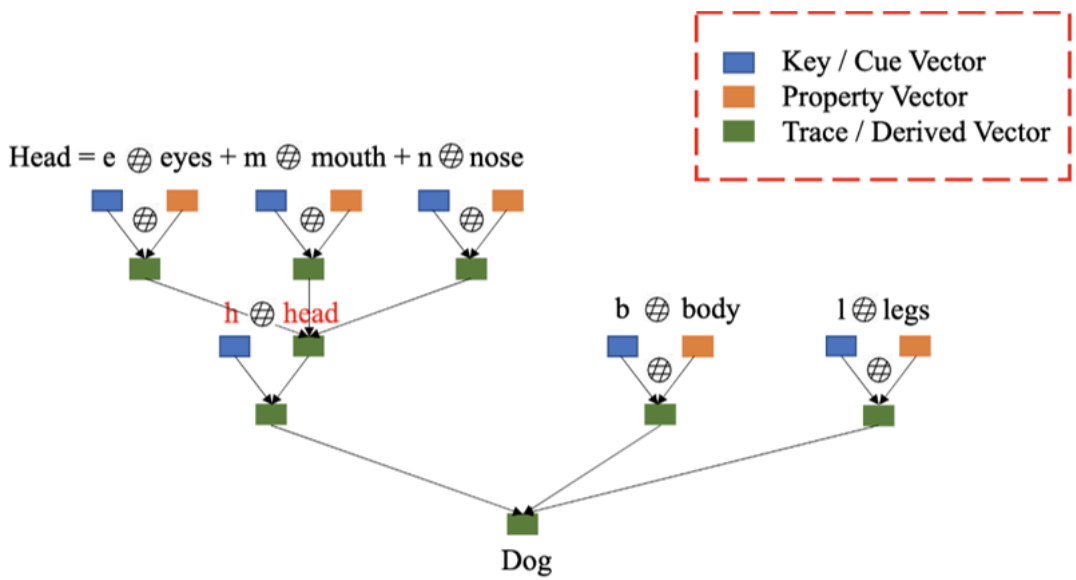


From: Ashwinkumar Ganesan et al., “Learning with Holographic Reduced Representations”, 2021, (Ga21)

Querying in Hierarchies

- Given dog and item memory M, you can ask e.g.:
  - What is the vector (or “pointer” to image) for the nose?

1. Decode the HD-vector $head$:
   - Compute $head \approx r_1 = h' \otimes dog$
   - Then clean up: retrieve $head$ from M (most similar vector to r_1)
2. Given $head$, decode $nose$: $nose \approx r_2 = n' \otimes head$, then clean up



From: Ashwinkumar Ganesan et al., “Learning with Holographic Reduced Representations”, 2021, (Ga21)

Data Structures for Sequences

- How can we store a sequence like $a, b, c, a, q, ...$ using VSAs?
- We could use records, with labels acting as positions:

$$v_1 \otimes a + v_2 \otimes b + v_3 \otimes c + v_4 \otimes a + v_5 \otimes q$$

- But it is more common and flexible to use permutations $\rho(x)$:

$$\rho^1(a) + \rho^2(b) + \rho^3(c) + \rho^4(a) + \rho^5(q)$$

- Note: We use the property of permutations that the result and argument are dissimilar:

$$\mathrm{sim}\big(x, \rho^k(x)\big) \approx 0$$

- … To disambiguate every element from each other, including double occurrence of a

Applications of VSAs

Applications in Deep Learning

- Recall that a sum of vectors is similar to each of the summands
  - => We can replace some operations for each vector in a set by a single operation on the vector sum
- This was exploited e.g. in the following papers
- Ashwinkumar Ganesan et al., **Learning with Holographic Reduced Representations**, 2021, (Ga21)
  - A “classical” deep NN outputs a single VSA-vector which represents a set of up to 670k labels
- Mohammad Mahmudul Alam et al., **Recasting Self-Attention with Holographic Reduced Representations**, ICML’23, (Al23)
  - Self-attention mechanism of a Transformer is replaced by VSA operations on bundled key-value pairs
  - Seemingly speeds up transformer training up to 370 times (???)

Duality of Symbolic and Sub-symbolic Learning

- A binding operation $y = f \otimes x$ can be considered as a mapping from a vector x to vector y, with f being a function representing the mapping
- Given x and y, we can immediately compute a suitable mapping f as $f = y \otimes x^{-1}$ (we use here the unbinding operation $\otimes x^{-1}$)
- This can be also used for complex transformations on VSA-encoded data structures, e.g. reasoning on logical formulas:
  1. $x_1 \rightarrow x_2$ VSA-encoded as $\mathrm{ante} \otimes x_1 + \mathrm{cons} \otimes x_2$
  2. $\neg x_1 \vee x_2$ VSA-encoded as $\mathrm{first} \otimes \mathrm{not} \otimes x_1 + \mathrm{sec} \otimes x_2$
  - We can compute the mapping f from the first to the second formula as $f = \mathrm{ante}^{-1} \otimes \mathrm{first} \otimes \mathrm{not} + \mathrm{cons}^{-1} \otimes \mathrm{sec}$
- Such **one-shot learning** (from one/few examples) is dual to **sub-symbolic** learning of f from data (e.g. via backpropagation)
- Largely unexplored area of **neural-symbolic** learning (Ne01, Ne02)

Other Applications of VSAs

- **Semantic Pointer Architecture Unified Network** (SPAUN) of Eliasmith et al. 2008, (El13)
  - A framework for building systems which resemble specific brain regions, such as the prefrontal cortex, basal ganglia, and thalamus
  - Uses VSAs (called there SPAs) for representing concepts and their combinations
  - A brain model with 2.5 million simulated neurons can recognize numbers, remember them, figure out numeric sequences, and write them down with a robotic arm
- VSAs have been used to implement **Finite State Machines**
- Also a **universal** (Turing-complete) **computer** should be possible with VSA (but not yet implemented)
  - Advantage: Learning from examples, not programming

Discussion: How can Inspiration from Neuroscience Help?

Is it Worth It?

- Major advances in AI in recent years come from computer science, and are only remotely inspired by neuroscience (if at all)
  - E.g. deep learning, algorithms in computer vision, generative AI, transformers, …

Is it worth it to look at neuroscience at all?

Persistent Problems of Current Machine Learning / AI

- On the level of observed phenomena, current ML/AI faces many problems compared to biological intelligence, e.g.:
  - No **one-shot learning**, i.e. learning from one/few examples
  - No **continual learning**, i.e. learning new concepts without forgetting old ones in arbitrary order
  - Lack of **interpretability**, i.e. ability to relate cause and effect in the model
  - Lack of **trustworthiness**:
    - models don’t know what they know, and what they don’t know
    - => Hallucinations of GPT3/4, and almost all ML models
  - Low robustness to **adversarial examples**, i.e. models are easily fooled by small perturbations of input data
  - Low **energy efficiency**
  - No theory: we don’t really know **why** e.g. GPT works so well

Hypothesis: Core Mechanisms of AI / ML are Fundamentally Insufficient

- It seems that the core mechanisms in current machine learning / AI face some fundamental problems
- **Possibly inspiration from neuroscience can help to overcome these problems**
- Example of such ML/AI mechanisms:
  - **Learning**: mostly via **backpropagation** with **end-to-end** training
  - **Information representation/storage**: as weights of neurons, with no control where information is stored in the model

There is Much More to Learn from Neuroscience

- Hierarchical Temporal Memory (HTM) by Numenta
- Sparse Distributed Memory (SDM) by Pentti Kanerva
- Unintended analogies to brain mechanisms
  - Transformer model architecture might be similar to brain structures
    - Caucheteux, C., Gramfort, A. & King, JR. Evidence of a predictive coding hierarchy in the human brain listening to speech, 2022, (Ca22)
- Hierarchical Markov chains advocated by Roy Kurzweil in his book “How to Create a Mind” (2012)

Thank you!

References

Al23 Mohammad Mahmudul Alam, Edward Raff, Stella Biderman, Tim Oates, and James Holt. 2023. Recasting self-attention with holographic reduced representations. ICML'23, Vol. 202, Article 23, 490-507, https://dl.acm.org/doi/10.5555/3618408.3618431

Ca22 Caucheteux, C., Gramfort, A. & King, JR. Evidence of a predictive coding hierarchy in the human brain listening to speech. Nat Hum Behav 7, 430–441 (2023). https://doi.org/10.1038/s41562-022-01516-2

Da17 Sanjoy Dasgupta et al. ,A neural algorithm for a fundamental computing problem. Science 358,793-796(2017). DOI:10.1126/science.aam9868

El13 Chris Eliasmith, How to Build a Brain: A Neural Architecture for Biological Cognition, Oxford University Press, 2013

Fe18 Johannes Felsenberg et al, Integration of parallel opposing memories underlies memory extinction, Cell 2018 Oct 18;175(3): 709-722.e15. doi: 10.1016/j.cell.2018.08.021.

Ga21 Ashwinkumar Ganesan et al., “Learning with Holographic Reduced Representations”, NeurIPS 2021, arxiv.org/abs/2109.02157

Hi22 G. Hinton, The Forward-Forward Algorithm: Some Preliminary Investigations, https://arxiv.org/abs/2212.13345, 2022.

Kl21 Denis Kleyko, Module 2 at Seminar “Computing with HD Vectors”, 2021, UC Berkeley, https://www.hd-computing.com/course-computing-with-high-dimensional-vectors#h.hh2loih13jrc

Li20 Lillicrap, T., Santoro, A., Marris, L., Akerman, C., and Hinton, G. E., Backpropagation and the brain. Nature Reviews Neuroscience, 21:335-346, 2020.

Mo20 Mehrab N. Modi, Yichun Shuai, Glenn C. Turner, The Drosophila Mushroom Body: From Architecture to Algorithm in a Learning Circuit, Annual Review of Neuroscience 2020 43:1, 465-484

Ne01 Jane Neumann. 2001. Holistic Processing of Hierarchical Structures in Connectionist Networks. Ph. D. Dissertation. The University of Edinburgh.

Ne02 Jane Neumann. 2002. Learning the systematic transformation of holographic reduced representations. Cogn. Syst. Res 3, 2 (2002), 227-235.

Ne20 Peer Neubert, Kenny Schlegel, Stefan Schubert, An Introduction to Vector Symbolic Architectures and Hyperdimensional Computing, ECAI 2020, https://www.tu-chemnitz.de/etit/proaut/workshops_tutorials/vsa_ecai20/rsrc/vsa_slides.pdf

Sh23 Yang Shen, Sanjoy Dasgupta, Saket Navlakha; Reducing Catastrophic Forgetting With Associative Learning: A Lesson From Fruit Flies. Neural Comput 2023; 35 (11): 1797-1819. doi: https://doi.org/10.1162/neco_a_01615

Sh18 Jaiyam Sharma and Saket Navlakha, Improving Similarity Search with High-dimensional Locality-sensitive Hashing, http://arxiv.org/abs/1812.01844, 2018